

	EAST WEST UNIVERSITY Department of Computer Science and Engineering Lab Assignment, Spring 2025
---	---

Course:	CSE430 - Software Testing and Quality Assurance Lab
Instructor:	Shartaz Sajid Nahid
Assignment:	Selenium WebDriver

Overview

Write a Python Notebook file that uses Selenium WebDriver to automate and test a live web application. The target site for this assignment is: <https://books.toscrape.com> .

Your notebook must be executable from top to bottom. Each task below should be a separate cell or section. Use markdown cells or comments in code to label each task.

Tasks

1. Set up a Chrome WebDriver instance and complete the following steps in a single code cell:
 - a. Navigate to <https://books.toscrape.com>
 - b. Print the page title and current URL.
 - c. Maximize the browser window.
 - d. Assert that the string "Books" appears in the page title. If the assertion fails, print a clear failure message.
 - e. Assert that the current URL starts with "https://".
 - f. Refresh the page, wait 2 seconds, then assert the title is unchanged after refresh.
2. On the homepage (<https://books.toscrape.com>), locate elements using four different strategies:
 - a. By.CLASS_NAME: Find all book title elements.
 - b. By.CSS_SELECTOR: Find all star-rating elements
 - c. By.XPATH: Find the "next" pagination button using an XPath expression.
 - d. By.TAG_NAME: Find all <article> elements on the page.

For each locator, print the count of elements found and at least one sample value (text or attribute). Add an assertion for each that the count is greater than zero.

3. Scrape book data from pages 1 through 4 of the catalogue. For each book found, extract: Title, Price, Star rating, and Availability.

Note: Star ratings are stored as class names (e.g., class="star-rating Three"), not as numbers. You will need to extract the second word from the class attribute.

After scraping all four pages:

- a. Print the total number of books collected. Assert it equals 80 (20 books per page).
 - b. Save the data to a CSV file named "books_data.csv" with columns: page, title, price, rating, availability.
 - c. Print the first 3 rows of collected data to confirm correctness.
4. This part requires interacting with individual book pages and validating content. Pick any 3 books from your scraped data in Task 3 (use their titles to find them). For each of the 3 books, navigate to its "detail page" and perform the following assertions:
- a. Assert the page title is not empty.
 - b. Assert the book title on the detail page matches/contains the title you scraped previously.
 - c. Assert the price on the detail page matches the price you scraped previously.
 - d. Assert that an Add to basket button is present on the page and is enabled.
 - e. Assert the availability text contains "In stock".
5. Use a helper function log(test_name, passed) to print PASS or FAIL for each assertion instead of letting failures crash the notebook. Print a final summary: how many assertions passed out of the total.

Submission Instructions

1. Submit a single .ipynb file where each code cell will contain a solution. File name format: "StudentID_SeleniumAssignment.ipynb"
 - The notebook must run cleanly from top to bottom. Run all cells before submitting.
 - Do not hardcode absolute paths. Use relative paths for file I/O (e.g, saving the CSV file in Task 3).
 - Add a markdown/Text cell at the top with your name, student ID, and date.
2. Submit a report. File name format: "StudentID_SeleniumReport.pdf".
 - The report should include: descriptions of how you solved the tasks (you may include short code snippets to describe), screenshots showing your browser automation running and your assertions passing.
 - Make sure to include a cover page with your name and id on it in the report.

